

Region	Created	Storage	Storage (average)	Simple Operations	Advanced Operations	Data Transfer
iad1	Just now	0 B	0 B/1 GB	0/10k	0/2k	0 B/10 GB

Quickstart

@vercel/blob.env.localCopy Snippet

```
1 import { put } from "@vercel/blob";
2
3 const { url } = await put('articles/blob.txt', 'Hello World!', { access: 'public' });
```

Browser

Getting Started

Projects

Settings

RESOURCES

Documentation

@vercel/blob

Getting Started

Get started using this Blob Storage by following the steps in one of these guides.

N

Next.js

S

SvelteKit

1 Connect to a Vercel project

Connect Project

If not yet done, start by connecting a project to your Blob store. This action injects the `BLOB_READ_WRITE_TOKEN` environment variable in your Vercel project.

If you don't already have a project, you can get started with a [Blob template](#).

2 Prepare your local project

Run `vercel link` and `vercel env pull` in your terminal so you have the latest environment variables for your project locally.

3 Install our package

Then run `npm install @vercel/blob` to install the Vercel Blob SDK.

4 Use in code

This example creates a page to upload files using the Next.js App Router and the server upload method: the file goes through your server first and then to Vercel Blob.

⚠️ Server uploads are limited to 4.5 MB, if you need more than that, see the [client uploads](#) quickstart.

src/app/avatar/upload/page.tsxTypeScript

```
1 'use client';
2
3 import type { PutBlobResult } from '@vercel/blob';
4 import { useState, useRef } from 'react';
5
6 export default function AvatarUploadPage() {
7   const inputFileRef = useRef<HTMLInputElement>(null);
8   const [blob, setBlob] = useState<PutBlobResult | null>(null);
9   return (
10     <>
11       <h1>Upload Your Avatar</h1>
12
13       <form
14         onSubmit={async (event) => {
15           event.preventDefault();
16
17           if (!inputFileRef.current?.files) {
18             throw new Error("No file selected");
19           }
20
21           const file = inputFileRef.current.files[0];
22
23           const response = await fetch(
24             `/api/avatar/upload?filename=${file.name}`,
25             {
26               method: 'POST',
27               body: file,
28             }
29           );
30
31           const newBlob = (await response.json()) as PutBlobResult;
32
33           setBlob(newBlob);
34         }
35       >
36       <input name="file" ref={inputFileRef} type="file" accept="image/jpeg, image/png, image/webp" required />
37       <button type="submit">Upload</button>
38     </form>
39     <div && {
40       <div>
41         Blob url: <a href={blob.url}>{blob.url}</a>
42       </div>
43     </div>
44   );
45 };
```

src/app/api/avatar/upload/route.ts

TypeScript

```
1 import { put } from '@vercel/blob';
2 import { NextResponse } from 'next/server';
3
4 export async function POST(request: Request): Promise<NextResponse> {
5   const { searchParams } = new URL(request.url);
6   const filename = searchParams.get('filename');
7
8   // ⚠️ The below code is for App Router Route Handlers only
9   const blob = await put(filename, request.body, {
10     access: 'public',
11   });
12
13   // Here's the code for Pages API Routes:
14   // const blob = await put(filename, request, {
15   //   access: 'public',
16   // });
17
18   return NextResponse.json(blob);
19 }
20
21 // The next lines are required for Pages API Routes only
22 // export const config = {
23 //   api: {
24 //     bodyParser: false,
25 //   },
26 // };
27
```